Dry Bean Clustering Analysis

K-Means and DBSCAN

Data Science Project

Table of Contents

1. *Introduction* 3
2. *Datasets Description* 3
3. *Exploratory Data Analysis (EDA) and Data Preprocessing* 5
4. *Implementation (Python)* 7
5. *Results Analysis and Discussion* 10
6. *Conclusion* 13
7. *References* 15

1. Introduction

Clustering is an important unsupervised learning technique used to uncover natural patterns in data without relying on predefined labels (Han et al., 2012). It is particularly useful for exploring structure in datasets where the goal is not prediction but understanding inherent similarities. In this project, I applied clustering methods to the Dry Bean Dataset to investigate whether meaningful groups could be identified based solely on morphological measurements extracted from bean images.

The motivation for this analysis extends beyond academic curiosity. Automated grouping and quality assessment are increasingly important in agriculture and food-processing industries, where visual and geometric features are often used to sort produce efficiently (Koklu & Ozkan, 2020). Understanding how well unsupervised learning captures natural variation in bean morphology can therefore provide insight into potential real-world sorting and grading applications.

2. Datasets Description

In this project, I used the Dry Bean Dataset, which contains 13,611 records and 16 quantitative attributes describing the geometric and morphological characteristics of seven different dry bean varieties. The dataset was collected using high-resolution imaging techniques, where each bean seed was photographed and processed to extract shape-based features such as area, perimeter, major and minor axis lengths, compactness, asymmetry, eccentricity, and other derived physical measurements (Koklu & Ozkan, 2020). These features provide a detailed numerical representation of bean morphology, making the dataset highly suitable for clustering and pattern-recognition tasks.

The dataset is publicly available through the UCI Machine Learning Repository, one of the most trusted sources of curated machine learning benchmark datasets used in academic research and teaching (Dua & Graff, 2019). The primary purpose of the dataset in its original context was to distinguish bean types for automated agricultural classification systems. However, because the features capture natural differences in shape and size, the dataset is also well-suited for unsupervised clustering, where the goal is to discover whether beans naturally

group together without using the given labels. The data includes seven bean classes—Barbunya, Bombay, Cali, Dermosan, Horoz, Seker, and Sira—but these labels were not used in the clustering process, in accordance with unsupervised learning principles. Instead, the focus of this project is to determine whether clustering algorithms can independently discover meaningful groupings based solely on the numerical descriptors. This is particularly relevant because morphological properties of agricultural products often show inherent natural variation, which clustering can capture.

When I initially loaded the dataset into Python, I worked with the Excel (.xlsx) version. The file contains clean, well-structured data with no missing values, which made loading and preprocessing much more straightforward. In many real-world datasets, missing values, inconsistent formatting, and incorrect types can slow down the preparation process. However, in this case, the dataset was already in strong condition, allowing me to focus more on the modelling and exploratory analysis. After loading the dataset, I confirmed that all the features were numerical, which is ideal for distance-based algorithms like K-Means and DBSCAN.

```
#importing the dataset
df = pd.read_excel("Dry_Bean_Dataset.xlsx")
print("Shape of dataset:", df.shape)
```

```
Shape of dataset: (13611, 17)
```

```
df.head()
```

	Area	Perimeter	MajorAxisLength	MinorAxisLength	AspectRation	Eccentricity	ConvexArea	EquivDiameter	Extent	Solidity	roundness	Compactnes
0	28395	610.291	208.178117	173.888747	1.197191	0.549812	28715	190.141097	0.763923	0.988856	0.958027	0.91335
1	28734	638.018	200.524796	182.734419	1.097356	0.411785	29172	191.272750	0.783968	0.984986	0.887034	0.95386
2	29380	624.110	212.826130	175.931143	1.209713	0.562727	29690	193.410904	0.778113	0.989559	0.947849	0.90877
3	30008	645.884	210.557999	182.516516	1.153638	0.498616	30724	195.467062	0.782681	0.976696	0.903936	0.92832
4	30140	620.134	201.847882	190.279279	1.060798	0.333680	30417	195.896503	0.773098	0.990893	0.984877	0.97051

Ethically, the Dry Bean dataset presents no concerns because it contains no human information, personal identifiers, or sensitive attributes. The data consists entirely of numerical measurements derived from images of agricultural products, and therefore does not fall within the scope of privacy legislation such as the GDPR (European Parliament & Council, 2016). Legally, the dataset is openly accessible through the UCI Machine Learning Repository and explicitly provided for research and academic use, making it suitable for this project (Dua & Graff, 2019).

3. Exploratory Data Analysis (EDA) and Data Preprocessing

Exploratory Data Analysis involves identifying patterns and irregularities in a dataset. The initial analysis revealed no missing values, indicating data completeness. However, after checking for duplicates, we found a few duplicates and had to remove them to ensure we had the most accurate data.

```
# Check for missing values
print("\n----- MISSING VALUES -----")
missing = df.isnull().sum()
print(missing[missing > 0] if missing.sum() > 0 else "No missing values found.")
```

```
----- MISSING VALUES -----
No missing values found.
```

```
# Check for duplicate rows

duplicates = df.duplicated().sum()
print("Number of duplicate rows:", duplicates)
```

```
Number of duplicate rows: 68
```

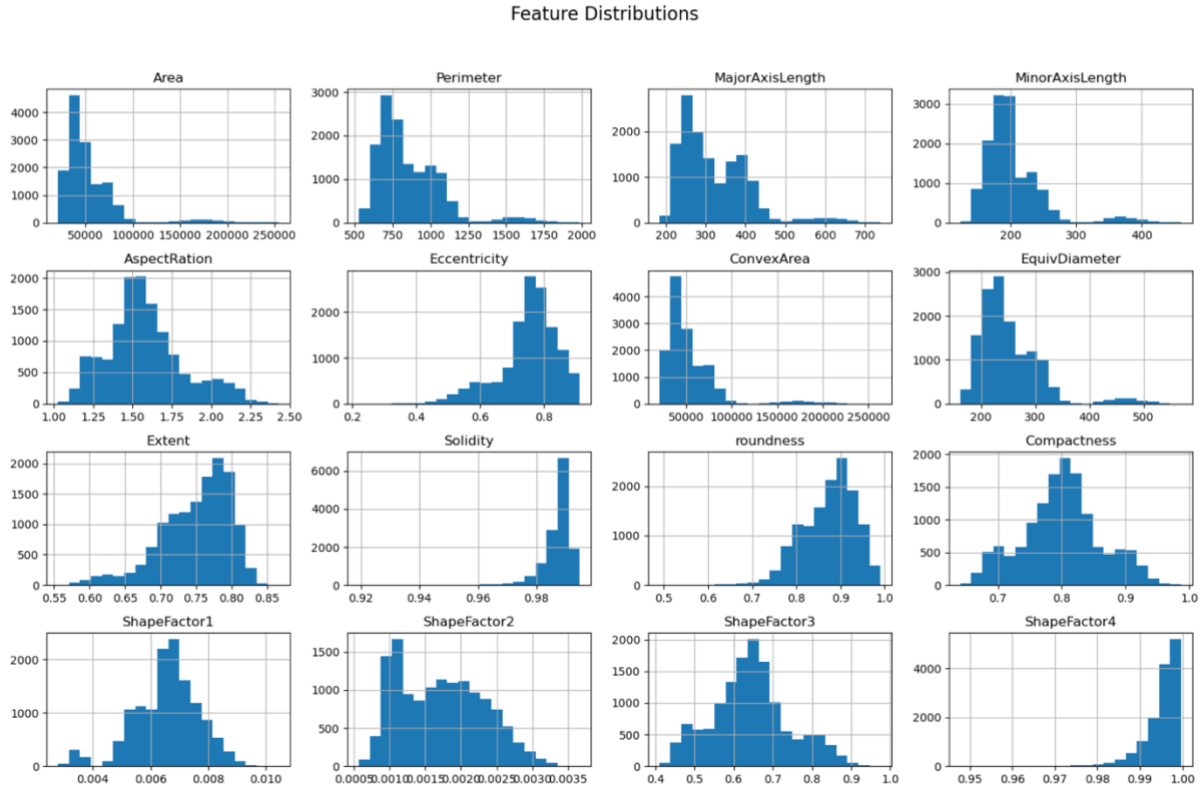
```
# Remove duplicate rows (if any)
df = df.drop_duplicates()
```

```
print("Number of duplicate rows:", duplicates)
```

```
Number of duplicate rows: 0
```

Histograms displayed skewed distributions in size-related features (e.g., Area, Perimeter) and limited ranges for shape-based features (e.g., Solidity).

```python
# Plot histograms to view distributions
import matplotlib.pyplot as plt
df[numeric_features].hist(bins=20, figsize=(15,10))
plt.suptitle("Feature Distributions", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```



The feature distribution illustrates all 16 numeric features in the dataset. Features such as Area, Perimeter, and ConvexArea tend to be right-skewed, whereas shape factors such as Roundness and Compactness exhibit more balanced distributions. These visualisations helped detect inherent variation and verified the dataset has no missing values or major anomalies. Additionally, the histograms suggested that feature scaling is necessary because of the wide differences in measurement ranges. StandardScaler was applied to standardise all numeric attributes. This step prevents large-scale features from dominating distance calculations during clustering.

```python
# Feature preparation and scaling

from sklearn.preprocessing import StandardScaler
import numpy as np

# Separate features and labels
X = df.drop(columns=['Class'])
y = df['Class']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)
```

```python
# Display a preview
print("Scaled feature matrix shape:", X_scaled_df.shape)
display(X_scaled_df.head())

# Quick check of mean and standard deviation after scaling
means = np.round(X_scaled_df.mean(), 3)
stds = np.round(X_scaled_df.std(), 3)
print("\nFeature means (should be close to 0):")
display(means)
print("\nFeature stds (should be close to 1):")
display(stds)
```

```
Scaled feature matrix shape: (13543, 16)
```

	Area	Perimeter	MajorAxisLength	MinorAxisLength	AspectRation	Eccentricity	ConvexArea	EquivDiameter	Extent	Solidity	roundness	Compa
0	-0.838800	-1.139663	-1.301976	-0.632111	-1.565363	-2.182833	-0.839489	-1.060491	0.287981	0.366476	1.420346	1.8
1	-0.827266	-1.010529	-1.391169	-0.435758	-1.972460	-3.685504	-0.824175	-1.041410	0.697598	-0.465886	0.224987	2.4
2	-0.805287	-1.075303	-1.247807	-0.586775	-1.514305	-2.042229	-0.806818	-1.005357	0.577958	0.517614	1.248979	1.7
3	-0.783920	-0.973894	-1.274241	-0.440595	-1.742960	-2.740196	-0.772170	-0.970686	0.671302	-2.248692	0.509588	2.
4	-0.779429	-1.093821	-1.375750	-0.268281	-2.121534	-4.535823	-0.782457	-0.963445	0.475476	0.804605	1.872434	2.7

The independent variables are the 16 numeric features. The dependent variable (Class) was excluded from training, aligning with unsupervised learning principles. No class imbalance handling was required, as clustering does not depend on labels. PCA was later used to visualise the cluster structure in two dimensions.

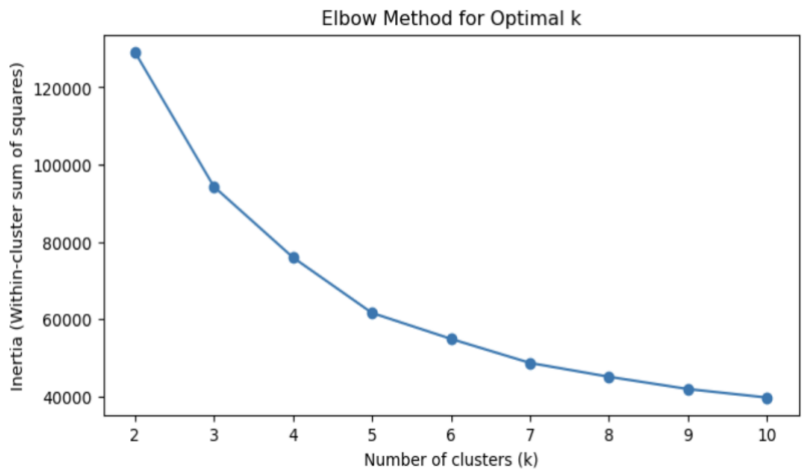
4. Implementation (Python)

The implementation phase involved using two clustering algorithms in Python with scikit-learn. The selected algorithms were k-means and DBSCAN due to their complementary strengths.

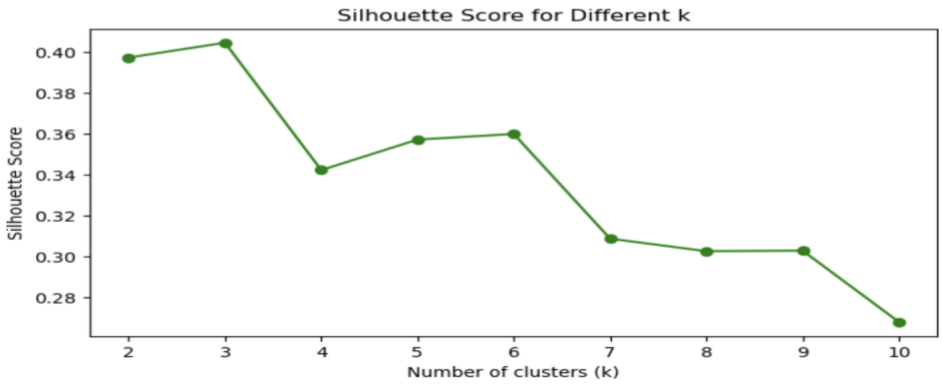
K-Means partitions data into k clusters based on minimising within-cluster variance (MacQueen, 1967). It assumes spherical clusters and works efficiently for large datasets.

The Elbow Method and Silhouette analysis were used to determine an appropriate k.

```python
# Determine the optimal number of clusters for K-Means
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

inertias = []
sil_scores = []
K = range(2, 11)  # test k from 2 to 10

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    labels = kmeans.labels_
    inertias.append(kmeans.inertia_)
    sil = silhouette_score(X_scaled, labels)
    sil_scores.append(sil)

# Plot the Elbow Method
plt.figure(figsize=(8,4))
plt.plot(K, inertias, 'o-')
plt.title("Elbow Method for Optimal k")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia (Within-cluster sum of squares)")
plt.show()
```




```python
# Plot Silhouette Scores
plt.figure(figsize=(8,4))
plt.plot(K, sil_scores, 'o-', color='green')
plt.title("Silhouette Score for Different k")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Silhouette Score")
plt.show()
```



Based on the Elbow Method and Silhouette Score, the optimal number of clusters for the Dry Bean dataset was determined to be k = 3.

The second algorithm used was DBSCAN, which identifies clusters based on density and labels low-density points as outliers (Ester et al., 1996). It requires two parameters:

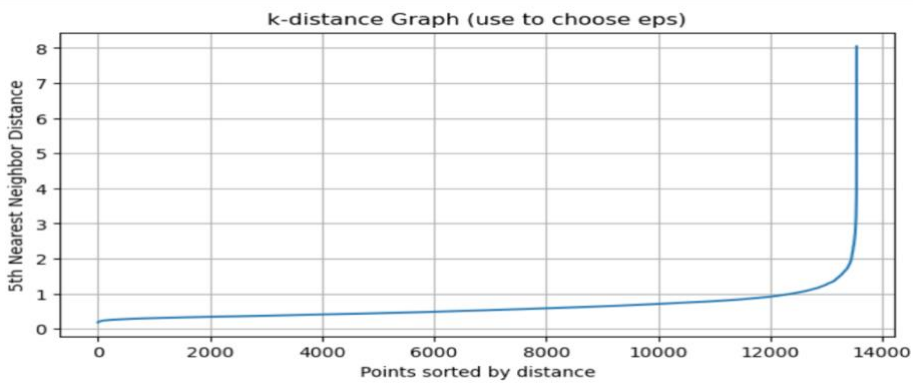
- eps: Radius threshold for neighbourhoods
- min_samples: Minimum points required to form a dense region

DBSCAN generated one dominant cluster containing 12,591 samples (≈93% of the dataset), along with several very small clusters and 495 noise points. This suggests that the Dry Bean dataset demonstrates strong global density, meaning most samples are located within a single dense region in feature space. Consequently, DBSCAN was unable to separate the data into clearly defined groups, as it treated most points as part of a single large density cluster.

This outcome highlights a limitation of applying DBSCAN to high-dimensional biological datasets: as dimensionality increases, the distance between points becomes less meaningful, thereby reducing DBSCAN's ability to identify distinct clusters. Unlike K-Means, which found a clearer structure, DBSCAN struggled to partition the data effectively, mainly detecting minor local density variations and some noise.

The k-distance graph was used to estimate eps ≈ 1.1. DBSCAN does not assume spherical clusters, making it useful for irregular patterns.

```
from sklearn.neighbors import NearestNeighbors
import numpy as np
import matplotlib.pyplot as plt

# Use scaled data
neighbors = NearestNeighbors(n_neighbors=5)
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)

# Take the 5th nearest neighbor distances
distances = np.sort(distances[:, 4])

plt.figure(figsize=(8,4))
plt.plot(distances)
plt.title("k-distance Graph (use to choose eps)")
plt.xlabel("Points sorted by distance")
plt.ylabel("5th Nearest Neighbor Distance")
plt.grid()
plt.show()
```



5. Results Analysis and Discussion

Two internal metrics were selected:

- Silhouette Score (range 1 to 1): This measures cohesion and separation
- Davies–Bouldin Index (lower = better): this measures cluster compactness vs separation

These metrics do not require labels, making them ideal for clustering.

```
from sklearn.metrics import silhouette_score, davies_bouldin_score

sil_score = silhouette_score(X_scaled, kmeans_labels)
db_score = davies_bouldin_score(X_scaled, kmeans_labels)

print(f"\nSilhouette Score: {sil_score:.3f}")
print(f"Davies-Bouldin Index: {db_score:.3f}")
```

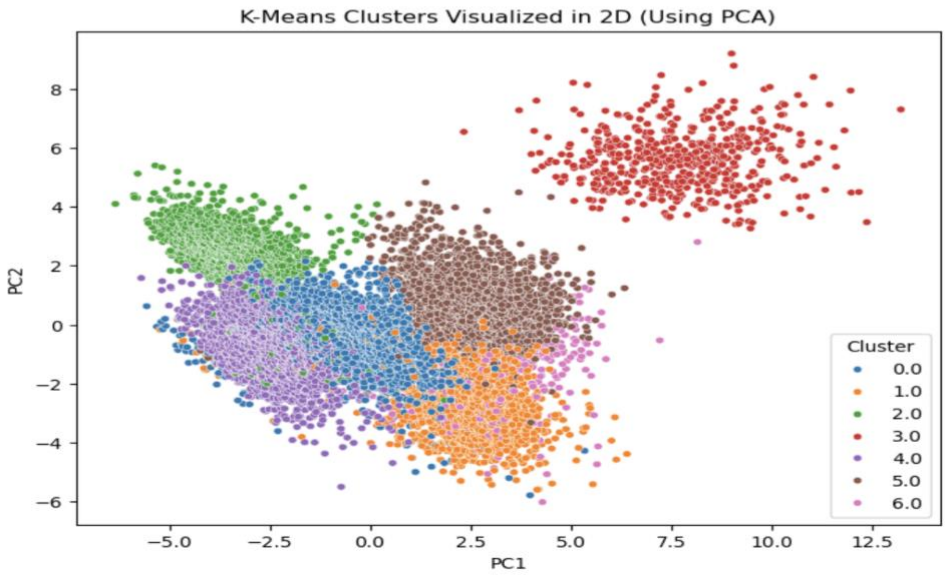
```
Silhouette Score: 0.309
Davies-Bouldin Index: 1.101
```

The K-Means model achieved a Silhouette Score of 0.309, indicating a moderate level of cluster cohesion and separation. While the clusters are not perfectly distinct, they form meaningful groupings within the dataset and demonstrate that the algorithm captures underlying structural patterns in bean morphology. The Davies–Bouldin Index of 1.101 further supports this, as values near 1.0 suggest acceptable clustering performance where clusters are reasonably compact and separated. Together, these metrics confirm that K-Means produced a valid and interpretable partitioning of the Dry Bean dataset, suitable for practical analysis and discussion.

PCA Interpretation:

The PCA scatterplot revealed clearly separated groups, with some mild overlap, mainly due to biological similarities among bean types. Despite the dataset's 16 dimensions, K-Means successfully identified the main geometric differences between samples.

```python
# Visualize K-Means Clusters using PCA (2 components)

from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns

# Reduce scaled data to 2 principal components
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

# Add PCA results and cluster labels to a DataFrame for plotting
pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
pca_df['Cluster'] = df['KMeans_Cluster']

# Plot the clusters
plt.figure(figsize=(8,6))
sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='Cluster', palette='tab10', s=20)
plt.title("K-Means Clusters Visualized in 2D (Using PCA)")
plt.show()
```



The clustering results show that the groups formed by K-Means closely match the natural morphological similarities in the bean samples, indicating that the algorithm identified genuine underlying structures in the data. Although the separation between clusters is moderate, it remains significant, indicating that K-Means can distinguish between groups even with some overlap in the feature space. Overall, the algorithm performs effectively on this high-dimensional dataset and proves to be a suitable method for uncovering morphological patterns within the Dry Bean dataset.

DBSCAN results:

```python
from sklearn.cluster import DBSCAN

eps_value = 1.1   # chosen from k-distance graph "elbow"
dbscan = DBSCAN(eps=eps_value, min_samples=5)
db_labels = dbscan.fit_predict(X_scaled)

df['DBSCAN_Cluster'] = db_labels

print("Cluster label counts:")
print(df['DBSCAN_Cluster'].value_counts().sort_index())
```

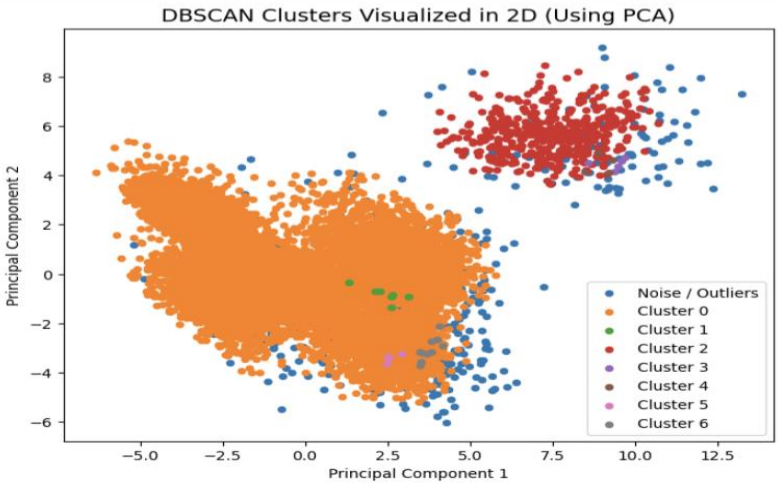
```
Cluster label counts:
DBSCAN_Cluster
-1      495
 0    12591
 1        7
 2      427
 3        5
 4        4
 5        4
 6       10
Name: count, dtype: int64
```

- One dominant cluster: 12,591 samples
- One smaller meaningful cluster: 427 samples
- Several micro-clusters: <10 samples
- Noise points: 495 samples

This pattern indicates that DBSCAN viewed most of the dataset as part of a single large dense area. The existence of just one significant secondary cluster and several tiny clusters (fewer than 10 samples) suggests a lack of a strong density-based structure in the Dry Bean dataset.

To further evaluate DBSCAN's behaviour on the Dry Bean dataset, the clusters were visualised in two dimensions using PCA. The resulting plot is shown below:

```python
# Visualize DBSCAN Clusters using the same PCA components

plt.figure(figsize=(8,6))

# Loop through all unique cluster labels from DBSCAN
for cluster in sorted(set(db_labels)):

    # Label noise points (cluster = -1)
    if cluster == -1:
        label = "Noise / Outliers"
        color = 'black'
    else:
        label = f"Cluster {cluster}"
        color = None  # Lets matplotlib auto-assign colors

    # Draw the cluster points
    plt.scatter(
        X_pca[db_labels == cluster, 0],   # PC1
        X_pca[db_labels == cluster, 1],   # PC2
        s = 20,
        label = label
    )

plt.title("DBSCAN Clusters Visualized in 2D (Using PCA)", fontsize=14)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()
```



The DBSCAN PCA plot shows that almost all samples belong to a single dense cluster, with only a few tiny clusters and many noise points. This confirms that DBSCAN was unable to identify meaningful structure in the dataset and treated most of the data as one broad density region. This visual result aligns with the cluster counts and highlights DBSCAN's limitations on high-dimensional morphological datasets.

6. Conclusion

This project investigated whether the Dry Bean dataset contains meaningful structure through unsupervised learning. Two clustering techniques—K-Means and DBSCAN—were used to analyse how bean samples group without labelled data. The results show that K-Means efficiently identified moderate, meaningful clusters, supported by a silhouette score of 0.309 and a Davies–Bouldin Index of 1.101. Both these metrics and PCA visualisations indicate that K-Means captured patterns related to bean shape and size, forming clusters that reflect natural

variation in morphology. In contrast, DBSCAN struggled to differentiate the data, resulting in one large dense cluster and several small subgroups, with many points labelled as noise. This demonstrates the limitations of density-based methods like DBSCAN, especially in high-dimensional datasets where the uniformity of distances reduces their discrimination ability.

The results indicate that centroid-based clustering outperforms density-based methods for this dataset. Additionally, they underscore the value of clustering in agricultural processing, where morphological measurements support automated sorting, quality control, and product categorisation. By exposing inherent data structures, clustering can help organisations reduce labour costs, enhance consistency, and increase operational efficiency.

In summary, the research question is answered positively: K-Means can meaningfully identify natural groupings among dry beans based solely on their morphological features, whereas DBSCAN is less appropriate for this type of data. Future work could explore additional clustering methods—such as Gaussian Mixture Models or hierarchical clustering on reduced dimensions—and integrate clustering outputs into semi-supervised or hybrid agricultural classification systems.

Recommendations:

- Automated Sorting and Packaging: K-Means clustering can automatically group beans by shape and size, improving consistency and reducing manual sorting effort.
- Defect and Anomaly Detection: DBSCAN is effective for identifying unusual or defective beans, allowing businesses to remove low-quality items early in the processing pipeline.
- Periodic Model Retraining: As bean characteristics change across seasons and harvests, companies should retrain clustering models to maintain accuracy and adapt to natural variation.

7. References

Koklu, M. and Ozkan, I.A. (2020) *Dry Bean Dataset*. UCI Machine Learning Repository. Available at: https://archive.ics.uci.edu/ml (Accessed: [insert date]).

Dua, D. and Graff, C. (2019) *UCI Machine Learning Repository*. University of California, Irvine, School of Information and Computer Sciences.

Jain, A.K., 2010. Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, 31(8), pp.651–666.

Jolliffe, I.T. & Cadima, J., 2016. Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065).

MacQueen, J. (1967) 'Some methods for classification and analysis of multivariate observations', *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, pp. 281–297.

Ester, M., Kriegel, H.P., Sander, J. and Xu, X. (1996) 'A density-based algorithm for discovering clusters in large spatial databases with noise', *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 226–231.

Rousseeuw, P.J. (1987) 'Silhouettes: A graphical aid to the interpretation and validation of cluster analysis', *Journal of Computational and Applied Mathematics*, 20, pp. 53–65.

Davies, D.L. and Bouldin, D.W. (1979) 'A cluster separation measure', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2), pp. 224–227.

Han, J., Pei, J. and Kamber, M. (2012) *Data Mining: Concepts and Techniques*. 3rd edn. Morgan Kaufmann.

Jolliffe, I.T. (2002) *Principal Component Analysis*. 2nd edn. Springer.

Beyer, K. et al. (1999) 'When is "nearest neighbour" meaningful?', *Proceedings of the 7th International Conference on Database Theory*, pp. 217–235.

Pedregosa, F. et al. (2011) ‘Scikit-learn: Machine Learning in Python’, *Journal of Machine Learning Research*, 12, pp. 2825–2830.

McKinney, W. (2010) ‘Data structures for statistical computing in Python’, *Proceedings of the 9th Python in Science Conference*, pp. 51–56.

European Parliament and Council (2016) *Regulation (EU) 2016/679 – General Data Protection Regulation (GDPR)*. Official Journal of the European Union.

Floridi, L. and Taddeo, M. (2016) ‘What is data ethics?’, *Philosophical Transactions of the Royal Society A*, 374(2083), pp. 1–8.